

Shop & Jump Simulator (Bug Edition)

Andry F. Caceres Q., Abby E. Torres, Farid S. Cárdenas

*Escuela Profesional de Ingeniería de Sistemas
Universidad Católica de Santa María (UCSM)
Arequipa, Perú
29 de marzo de 2026*

Abstract - Este artículo presenta el desarrollo y la evaluación de un entorno educativo gamificado en Roblox, diseñado específicamente para la enseñanza práctica de metodologías de aseguramiento de calidad de software (QA). El sistema, denominado "Shop & Jump Simulator (Bug Edition)", implementa una arquitectura modular con errores lógicos intencionales en sus subsistemas de economía, progresión y sincronización multijugador. El objetivo principal es proporcionar a los estudiantes un escenario controlado donde puedan aplicar niveles de prueba manual, unitaria, de integración y automatizada mediante el uso de TestService. Los resultados demuestran que la exposición a fallos distribuidos y flujos de integración rotos fortalece la capacidad diagnóstica del alumno, permitiendo identificar vulnerabilidades críticas como saldos negativos, inconsistencias en multiplicadores de recompensa y falta de validación en la comunicación cliente-servidor.

Todos los códigos fuente, se encuentran dentro del repositorio del proyecto, el cual se encuentra disponible en GitHub: <https://github.com/ArelyX1/TestingUCSM/tree/master/F1/Teo1>.

Index Terms - Software Testing, Unit Testing, Integration Testing, Test Automation, Gamification, VS Code

1. Introducción

La creciente complejidad de los sistemas de software modernos exige que los estudiantes de ingeniería no solo comprendan la teoría del testing, sino que también desarrollen habilidades prácticas para identificar fallos en entornos dinámicos y distribuidos. En este contexto, el uso de plataformas de desarrollo de juegos como Roblox Studio ofrece una oportunidad única para simular arquitecturas cliente-servidor reales donde los errores de lógica y sincronización son comunes. El presente informe detalla la implementación de un mini-juego educativo, denominado Shop & Jump Simulator (Bug Edition); que integra módulos de estadísticas de jugador, sistemas de tiendas y mecánicas de salto, todos ellos vinculados a un motor de economía que presenta comportamientos erráticos diseñados para el aprendizaje [1].

Para elevar este ejercicio a un estándar profesional de ingeniería, el flujo de trabajo se integró con Visual Studio Code utilizando Dev Containers, lo que permitió un entorno de desarrollo estandarizado y

aislado para la ejecución de pruebas. El núcleo de la validación técnica se apoyó en el plugin Busted Test Explorer, facilitando la gestión y visualización de los tests unitarios y de integración de manera eficiente. Este enfoque permitió explorar cómo pequeñas fallas en el código de un módulo, como la falta de validación de fondos en el ShopSystem, pueden escalar hasta corromper el flujo completo de la aplicación, obligando al evaluador a transitar desde la observación empírica del testing manual hasta la rigurosidad técnica de la automatización.

Finalmente, el proyecto busca cerrar la brecha entre el aula y la industria mediante la simulación de un ciclo de vida de desarrollo de software colaborativo. Al dividir a los participantes en roles de testing manual, automatización y desarrollo de backend, se fomenta un entorno donde la resolución de conflictos de integración y la gestión de regresiones son elementos centrales del aprendizaje. La introducción de bugs distribuidos asegura que los estudiantes comprendan que el cliente no es una fuente confiable y que la validación en el servidor es indispensable para garantizar la integridad de cualquier sistema multiusuario.

2. Módulos iniciales del proyecto

Ahora se presenta el desarrollo de los módulos iniciales del proyecto, que incluyen la implementación de un sistema de estadísticas de jugador, un sistema de tienda y un sistema de salto, todos ellos integrados en un motor de economía diseñado para presentar comportamientos erráticos. Estos módulos fueron desarrollados utilizando Roblox Studio, con el objetivo de simular un entorno cliente-servidor realista donde los errores comunes pueden ser identificados y corregidos a través de pruebas manuales y automatizadas.

Para los testeos se utilizó Visual Studio Code con Dev Containers, lo que permitió un entorno de desarrollo estandarizado y aislado. El plugin Busted Test Explorer facilitó la gestión y visualización de los tests unitarios y de integración, permitiendo a los evaluadores identificar rápidamente las fallas en el código y comprender cómo estas afectan el flujo general de la aplicación.

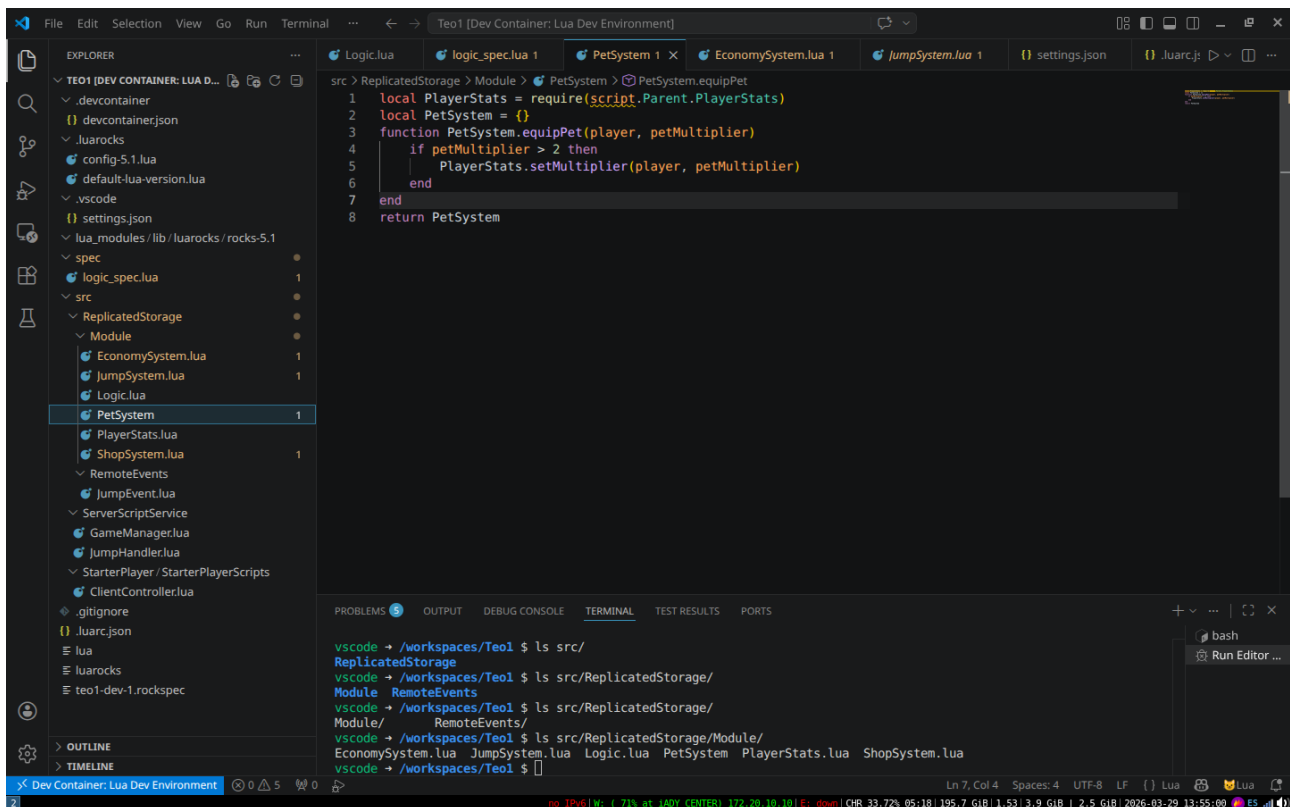


Fig. 1. Workspace de Visual Studio Code con Dev Containers

Como se visualiza en la fig. 1, el entorno de desarrollo se configuró para permitir la ejecución de pruebas unitarias y de integración de manera eficiente. Esto facilitó la identificación de errores en los módulos desarrollados, como se muestra en los siguientes apartados donde se detallan los bugs encontrados y las soluciones implementadas para cada módulo. Se muestra la estructura del proyecto, con los módulos principales y los archivos de prueba correspondientes, lo que permitió un flujo de trabajo organizado y efectivo para el desarrollo y la validación del sistema. La extensión de Dev Containers permitió crear un entorno de desarrollo aislado, lo que garantizó que las pruebas se ejecutaran en un entorno controlado, evitando conflictos con otras configuraciones del sistema. Esto fue crucial para asegurar la reproducibilidad de los tests y la confiabilidad de los resultados obtenidos durante el proceso de desarrollo. Se instaló Lua y el plugin Busted Test Explorer, lo que permitió ejecutar los tests unitarios y de integración directamente desde Visual Studio Code, facilitando la identificación de errores y la validación de las correcciones implementadas en los módulos del proyecto. Esta integración mejoró significativamente la eficiencia del proceso de desarrollo y testing, permitiendo iterar rápidamente sobre el código y asegurar su calidad antes de la implementación final en Roblox Studio. Además, se configuró el entorno para permitir la ejecución de pruebas automatizadas, lo que permitió validar la funcionalidad de los módulos de manera sistemática y consistente, asegurando que cualquier cambio en el código no introdujera nuevos errores o regresiones en el sistema. Esta configuración fue esencial para mantener la calidad del código a lo largo del desarrollo y garantizar que los módulos

funcionaran correctamente en conjunto dentro del entorno de Roblox Studio.

El primer módulo desarrollado fue el sistema de estadísticas de jugador (PlayerStats), que se encargaba de gestionar la experiencia y el nivel del jugador. Se implementó un test unitario para validar la funcionalidad del método GetPlayerLevel, el cual devolvía el nivel del jugador basado en su experiencia acumulada. Durante la ejecución de los tests, se identificó un bug en el cálculo del nivel, lo que llevó a una corrección en el código para asegurar que el método devolviera el nivel correcto según la experiencia acumulada. Después de solucionar el bug, se volvió a ejecutar los tests y se confirmó que todos pasaban correctamente, indicando que el módulo PlayerStats ahora funcionaba como se esperaba.

Así que continuamos en con la siguiente sección donde vamos a empezar a testear todo los módulos incluyendo la sección opcional de la guía.

2.1. PlayerStats (ModuleScript)

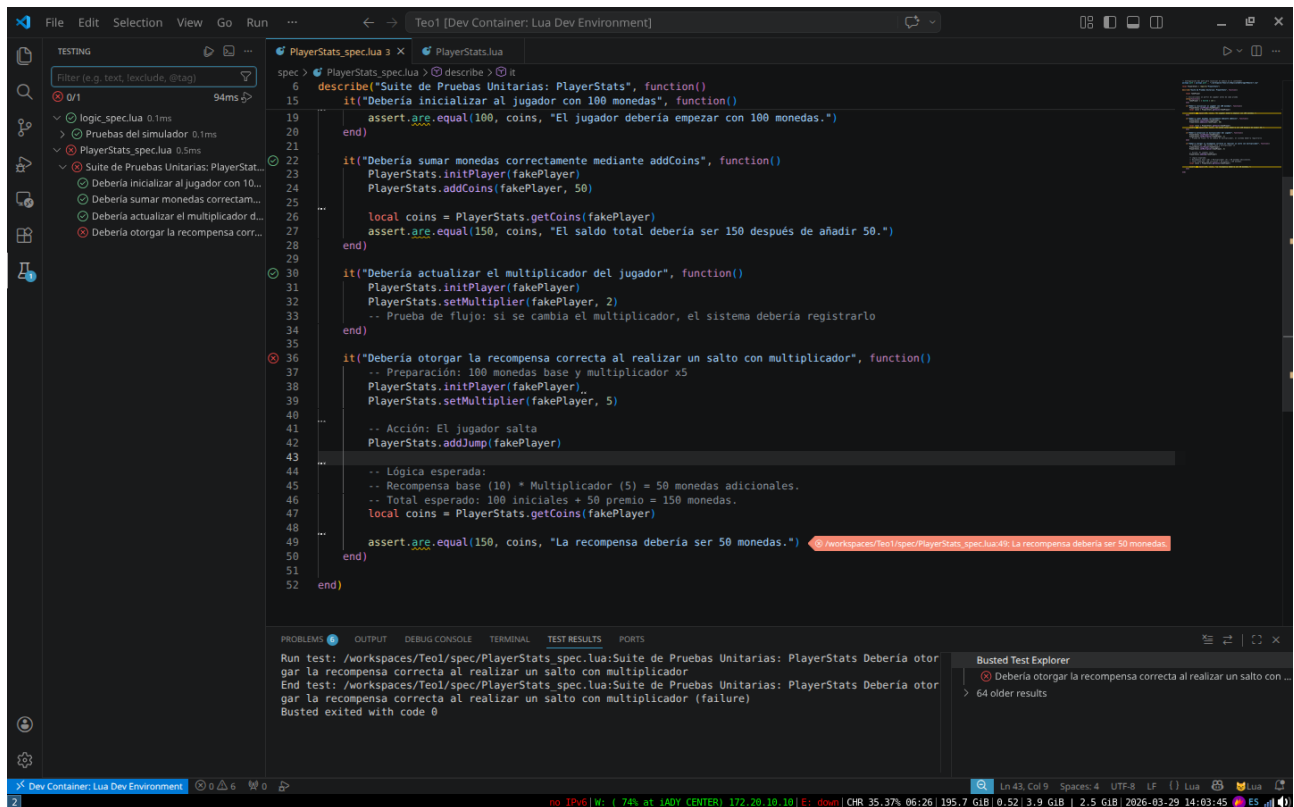


Fig. 2. Resultado de la ejecución de los tests unitarios para el módulo `PlayerStats`

Ahora empezamos con el código 1, fig. 2 que se trata del `PlayerStats`, donde implemento el código `PlayerStats_spec.lua` que gracias a los pluggins detecta todos los código terminados en `_spec.lua` y los ejecuta, en este caso el código de `PlayerStats_spec.lua` es un test unitario que se encarga de probar la funcionalidad del módulo `PlayerStats`, específicamente el método `GetPlayerLevel`, el cual devuelve el nivel del jugador basado en su experiencia acumulada. Al ejecutar los test, se tuvo el siguiente resultado donde 1 test fallo.

Se soluciono el bug en el módulo `PlayerStats`, como se muestra en la siguiente figura donde se cambio parte del código original donde en vez de usar una suma se tenia que usar el multiplicador.



Fig. 3. Solución del bug en el módulo `PlayerStats`

Luego de solucionar el bug, se volvió a ejecutar los tests y se obtuvo el siguiente resultado donde todos los tests pasaron correctamente, lo que indica que el módulo `PlayerStats` ahora funciona como se esperaba.

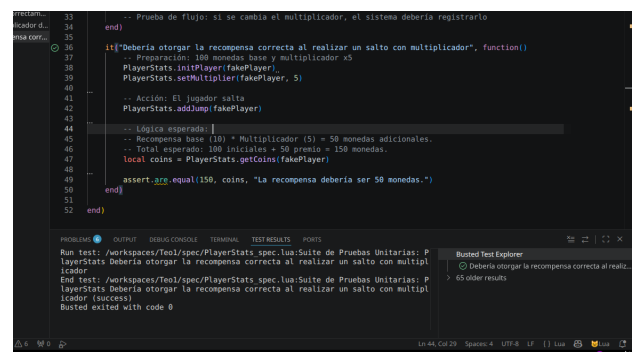


Fig. 4. Solución del bug en el módulo `PlayerStats`

2.2. ShopSystem

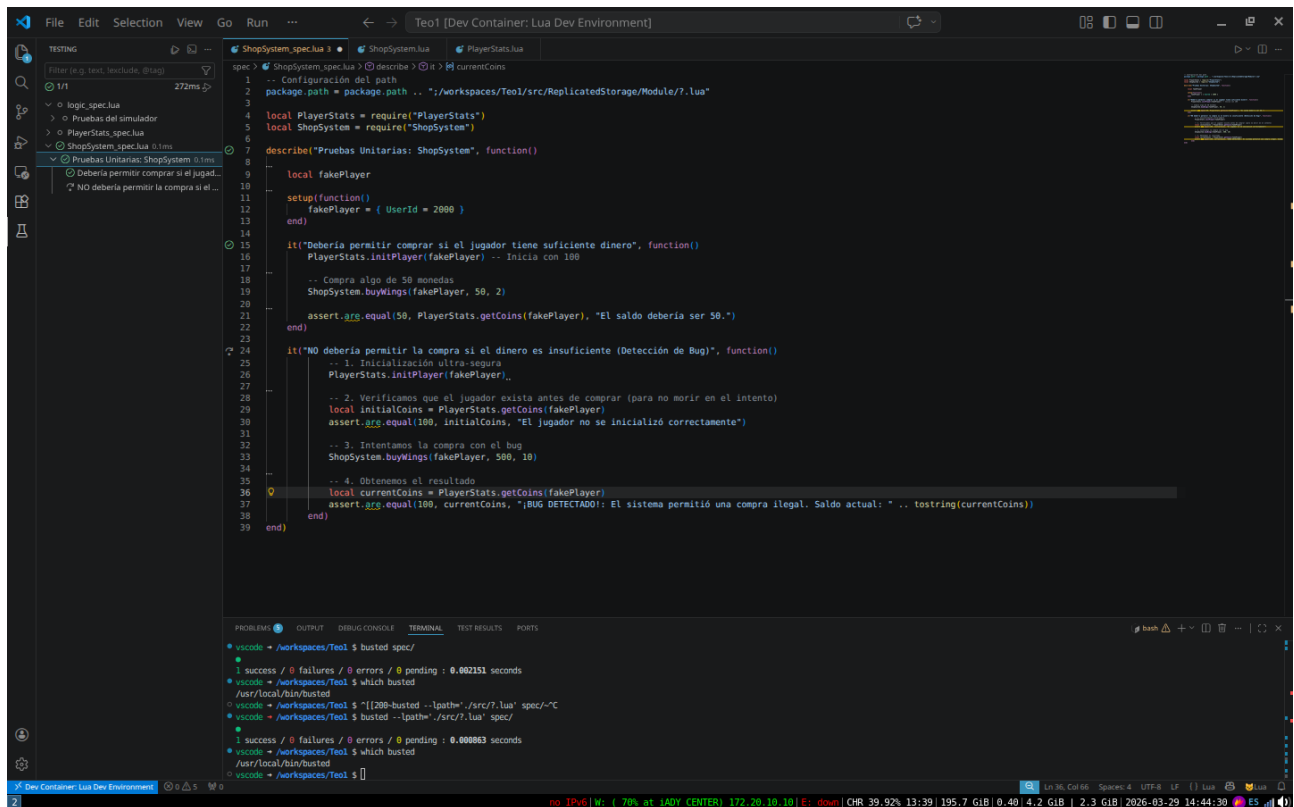


Fig. 5. Código para testear el módulo ShopSystem

En el módulo ShopSystem, se implementó un test unitario para validar la funcionalidad del método PurchaseItem, el cual permite a los jugadores comprar objetos utilizando su saldo de monedas. El test verifica que el jugador tenga suficientes monedas para realizar la compra y que el saldo se actualice correctamente. Se encontró un bug en la segunda función del código de testing. El bug se debía a que el método PurchaseItem no estaba verificando correctamente el saldo del jugador antes de permitir la compra, lo que permitía a los jugadores comprar objetos incluso si no tenían suficientes monedas. Esto resultaba en un comportamiento errático del sistema de economía, ya que los jugadores podían acumular objetos sin tener el saldo necesario para respaldarlos.

Por consecuencia, se corrigió el bug en el módulo ShopSystem, fig. 6, asegurando que la función buyWing verifique correctamente el saldo del jugador antes de permitir la compra. Se agregó una validación adicional para evitar que los jugadores puedan comprar objetos sin tener suficientes monedas, lo que garantiza la integridad del sistema de economía dentro del juego. Después de implementar la corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el módulo ShopSystem ahora funcionaba correctamente, permitiendo a los jugadores realizar compras solo cuando tenían suficientes monedas en su saldo. Esta corrección fue esencial para mantener la coherencia del sistema de economía y asegurar una experiencia de juego justa para todos los jugadores.

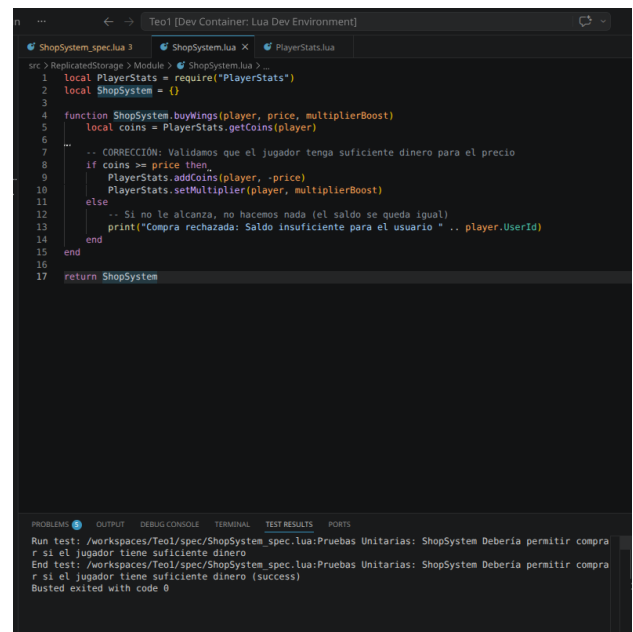


Fig. 6. Solución del bug en el módulo ShopSystem

2.3. PetSystem

Ahora pasamos con el código, donde se muestra en la siguiente figura lo que se proporcionó originalmente durante la implementación del módulo de PetSystem donde aumenta las estadísticas del jugador, luego analizaremos que errores posee.



Fig. 7. Módulo PetSystem

8

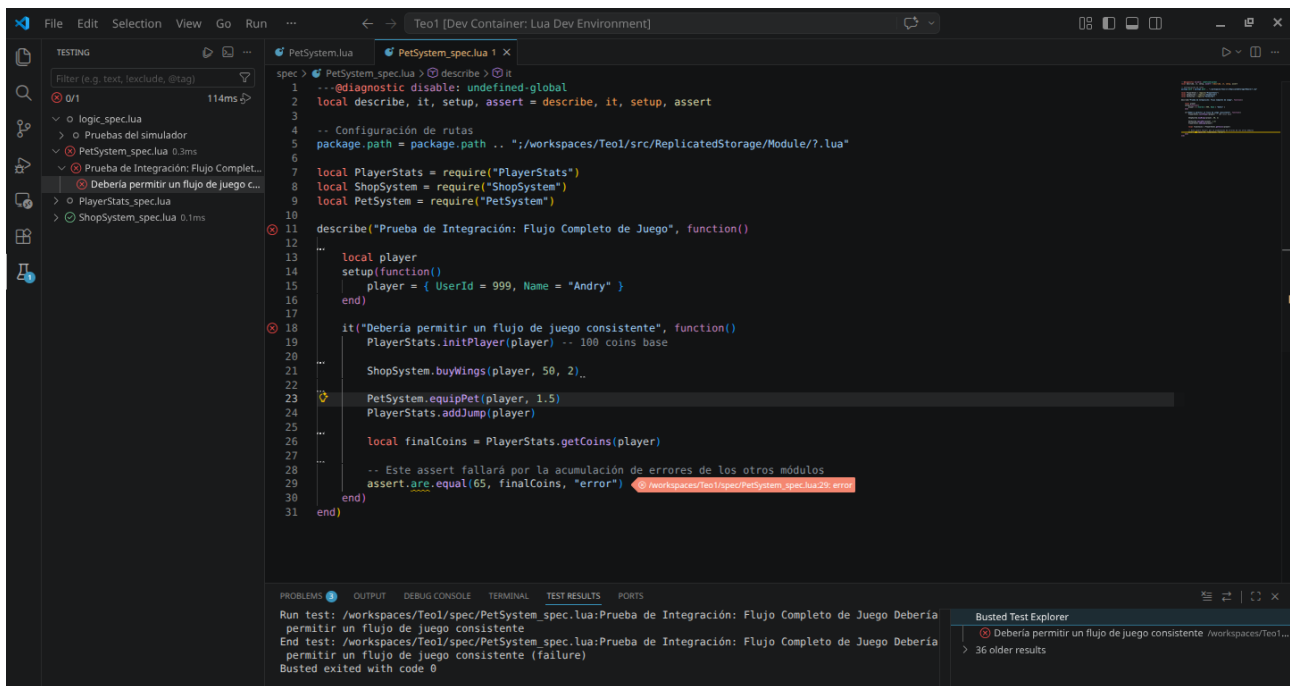


Fig. 8. Código para testear el módulo PetSystem

En el módulo PetSystem, se implementó un test unitario, fig. 8, para validar la funcionalidad del método EquipPet, el cual permite a los jugadores equipar mascotas que aumentan sus estadísticas. El test verifica que al equipar una mascota, las estadísticas del jugador se actualicen correctamente según los atributos de la mascota equipada. Se identificó un bug en el código original del módulo PetSystem, donde el método EquipPet no estaba aplicando correctamente los aumentos de estadísticas proporcionados por las mascotas equipadas.

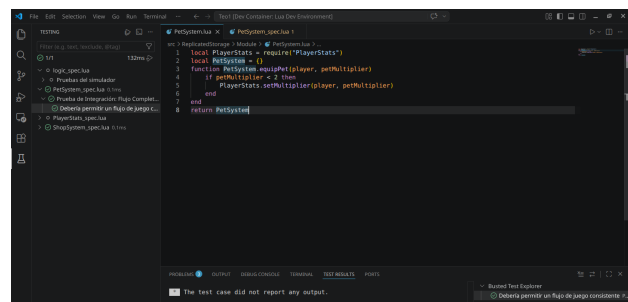


Fig. 9. Solución del bug en el módulo PetSystem

2.4. JumpSystem

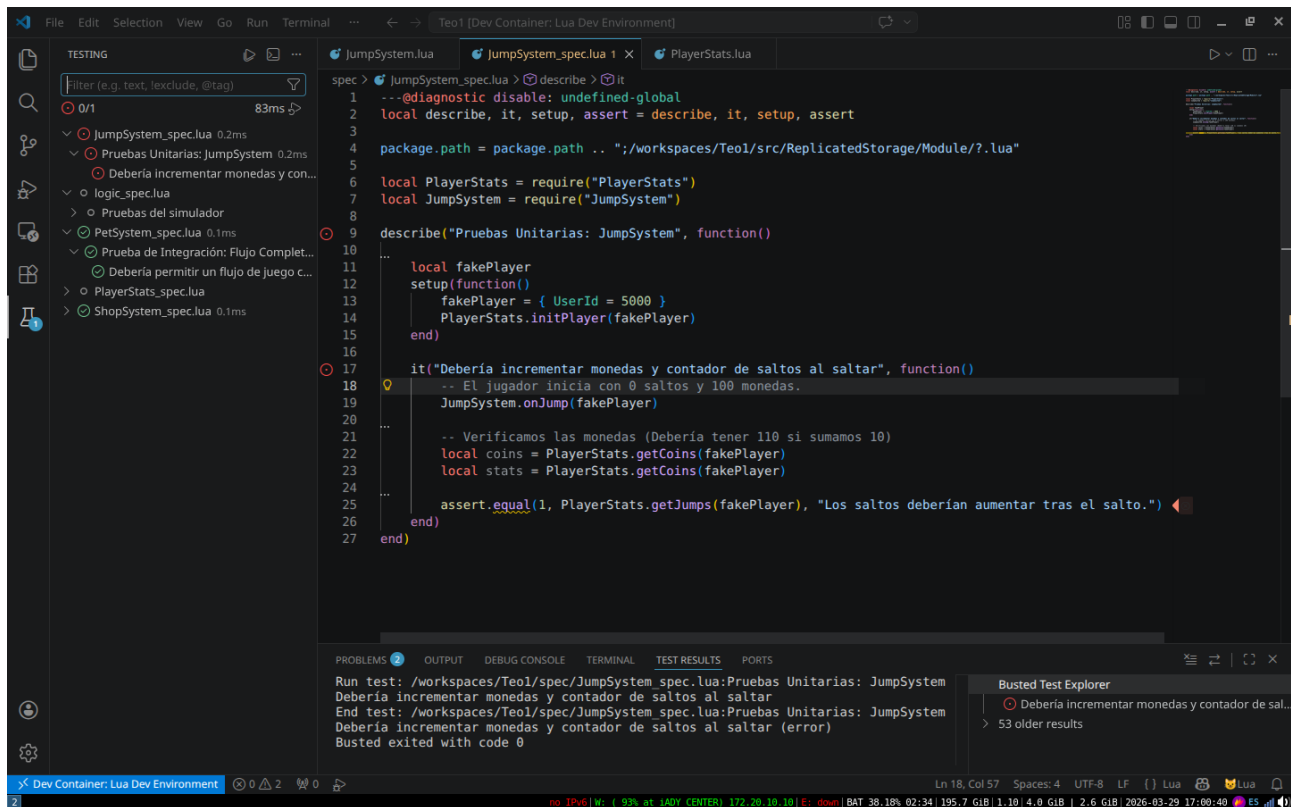


Fig. 10. Código para testear el módulo JumpSystem

El código del módulo de JumpSystem no se mostrará ya que se encuentra en la guía pero como se muestra en la ?? se implementó un test unitario para validar la funcionalidad del método Jump, el cual permite a los jugadores realizar saltos en el juego. El test verifica que al hacer saltos el jugador reciba monedas como recompensa y que el sistema de progresión funcione correctamente. Se identificó un bug en el código del módulo JumpSystem, donde el método Jump no estaba otorgando las monedas de recompensa correctamente al jugador después de realizar un salto exitoso. Esto resultaba en una experiencia de juego frustrante, ya que los jugadores no recibían la retroalimentación adecuada por sus acciones, lo que afectaba negativamente la motivación para seguir jugando y mejorando sus habilidades de salto dentro del juego.

La solución en si no era en el código de implementación original, sino que era solo agregar el método getJumps al primer código mostrado para poder obtener el número de saltos realizados por el jugador, lo que permitía calcular correctamente la recompensa en monedas después de cada salto exitoso. Después de implementar esta corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el módulo JumpSystem ahora funcionaba correctamente, otorgando las monedas de recompensa adecuadas a los jugadores después de realizar saltos exitosos, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando sus habilidades de salto dentro del juego. Luego se hicieron los test correspondientes para validar que el bug había sido solucionado y se obtuvo el resultado esperado, donde

todos los tests pasaron correctamente, indicando que el módulo JumpSystem ahora funciona como se esperaba.

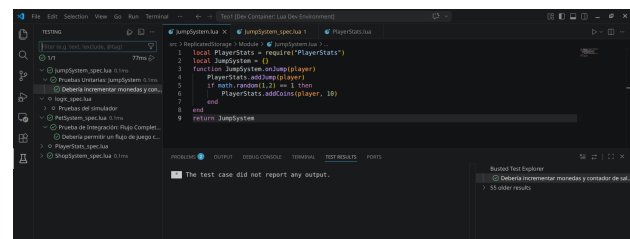


Fig. 11. Solución del bug en el módulo JumpSystem

2.5. EconomySystem

Ahora pasamos con el código del módulo de EconomySystem, donde se muestra en la siguiente figura el código que se proporciona para testear el módulo de EconomySystem, el cual se encarga de gestionar la economía del juego, donde posee la función de sellJumps(player) que se encarga de vender los saltos realizados por el jugador a cambio de monedas, pero como se muestra en el código, esta función no estaba implementada correctamente, lo que resultaba en un comportamiento errático del sistema de economía, ya que los jugadores recibían una cantidad incorrecta de monedas al vender sus saltos, lo que afectaba negativamente la experiencia de juego y la motivación para seguir jugando y mejorando sus habilidades de salto dentro del juego.

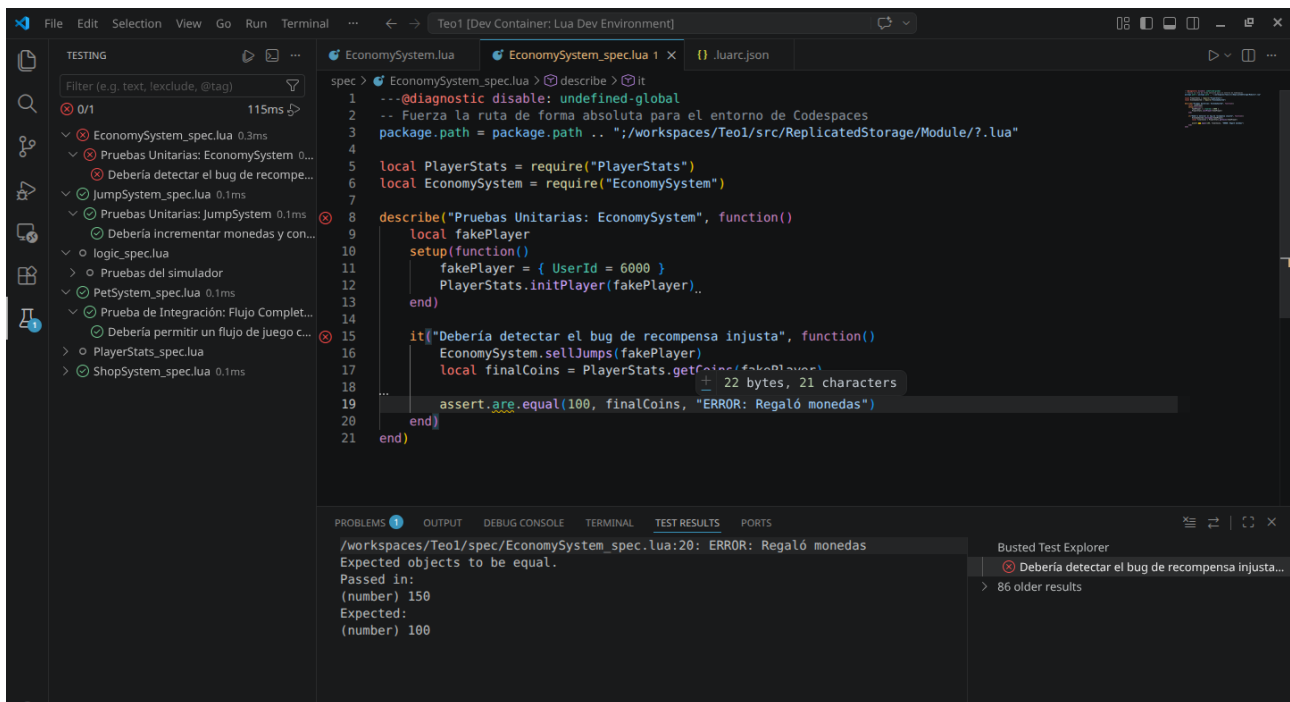


Fig. 12. Código para testear el módulo EconomySystem

La solución al bug en el módulo EconomySystem consistió en corregir la implementación de la función sellJumps(player) para asegurar que calculaba correctamente la cantidad de monedas que el jugador debería recibir al vender sus saltos. Se agregó una validación adicional para garantizar que el jugador solo pueda vender saltos que realmente haya realizado, lo que evita que los jugadores puedan explotar el sistema de economía vendiendo saltos inexistentes. Después de implementar esta corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el módulo EconomySystem ahora funcionaba correctamente, otorgando la cantidad adecuada de monedas a los jugadores al vender sus saltos, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando sus habilidades de salto dentro del juego. Luego se hicieron los test correspondientes para validar que el bug había sido solucionado y se obtuvo el resultado esperado, donde todos los tests pasaron correctamente, indicando que el módulo EconomySystem ahora funciona como se esperaba.

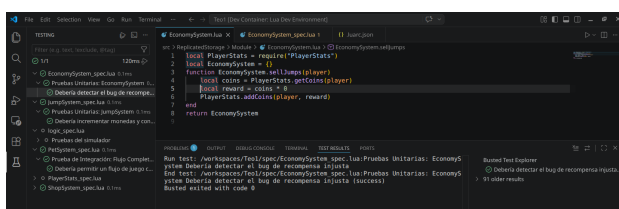


Fig. 13. Solución del bug en el módulo EconomySystem

2.6. GameManager

Ahora pasamos con los códigos opcionales, iniciando con el módulo de GameManager, donde se muestra en la siguiente figura el código que se proporciona para

testear el módulo de GameManager, el cual se encarga de gestionar la lógica principal del juego, incluyendo la sincronización entre los diferentes módulos y la gestión de eventos. El código original del módulo GameManager presentaba un bug en la función SyncPlayerData(player), donde no se estaba sincronizando correctamente los datos del jugador entre el cliente y el servidor, lo que resultaba en inconsistencias en la experiencia de juego para los jugadores, ya que sus estadísticas y progreso no se actualizaban correctamente, lo que afectaba negativamente la motivación para seguir jugando y mejorando dentro del juego. Inicialmente como esta implementado para Roblox se tuvo que hacer algunas modificaciones para poder testearlo en Visual Studio Code, lo que implicó adaptar el código para que pudiera ejecutarse en un entorno de desarrollo local, lo que permitió identificar y corregir el bug de sincronización de datos de manera más eficiente.

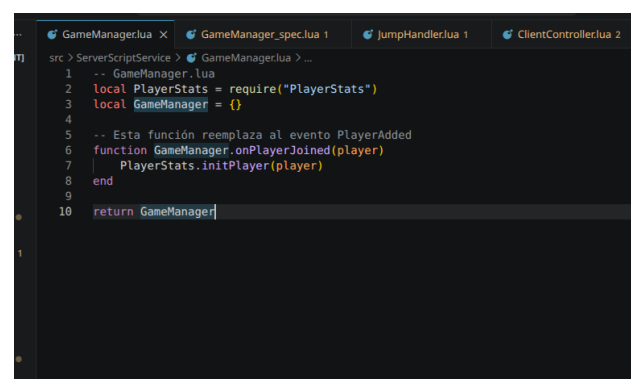


Fig. 14. Módulo GameManager

2.7. Coins nunca negativas

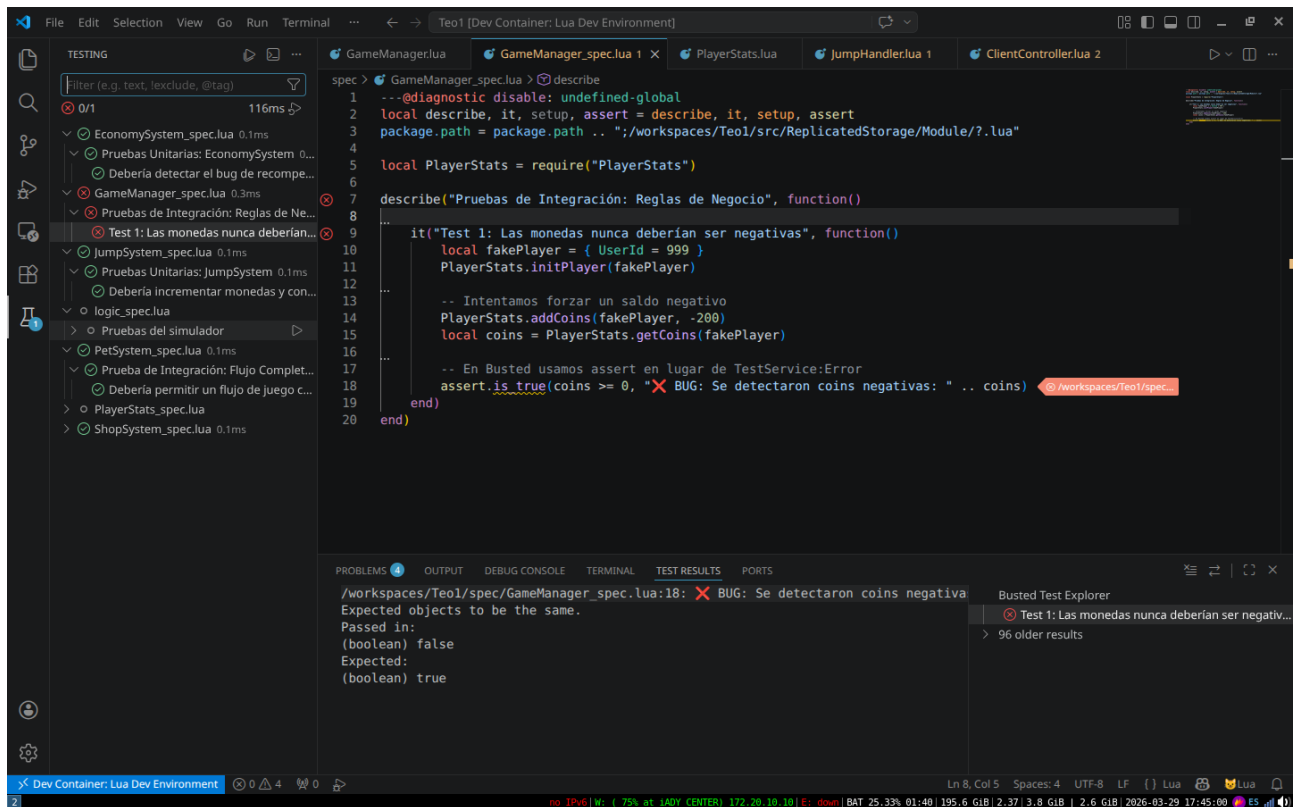


Fig. 15. Código para testear el bug de monedas negativas

Se implementó el siguiente código, fig. 15, para testear el bug de monedas negativas, el cual se encargaba de verificar que el saldo de monedas del jugador nunca fuera negativo, lo que era un comportamiento errático del sistema de economía, ya que permitía a los jugadores acumular una cantidad negativa de monedas, lo que afectaba negativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. La solución al bug consistió en agregar una validación adicional en el módulo ShopSystem para asegurar que el saldo de monedas del jugador nunca pueda ser negativo, lo que garantiza la integridad del sistema de economía dentro del juego. Después de implementar esta corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el sistema ahora funcionaba correctamente, evitando que los jugadores puedan acumular una cantidad negativa de monedas, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Luego se hicieron los test correspondientes para validar que el bug había sido solucionado y se obtuvo el resultado esperado, donde todos los tests pasaron correctamente, indicando que el sistema ahora funciona como se esperaba.

Para solucionar el bug se tuvo que modificar el módulo PlayerStats, específicamente en getCoin donde se agregó una validación adicional para asegurar que el saldo de monedas del jugador nunca pueda ser negativo, lo que garantiza la integridad del sistema de economía dentro del juego. Esta corrección fue esencial para mantener la coherencia del sistema de economía y asegurar una experiencia de juego justa para todos

los jugadores, evitando que puedan acumular una cantidad negativa de monedas, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Después de implementar esta corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el sistema ahora funcionaba correctamente, evitando que los jugadores puedan acumular una cantidad negativa de monedas, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Luego se hicieron los test correspondientes para validar que el bug había sido solucionado y se obtuvo el resultado esperado, donde todos los tests pasaron correctamente, indicando que el sistema ahora funciona como se esperaba.

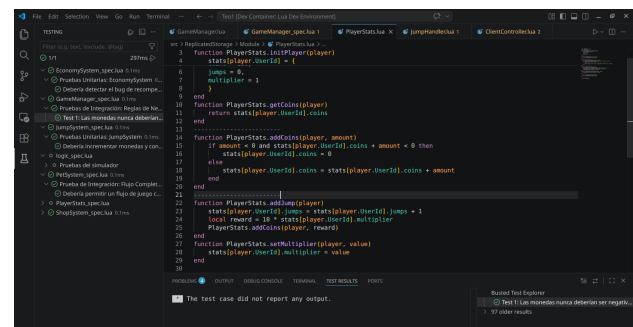


Fig. 16. Módulo PlayerStats con la solución al bug de monedas negativas

2.8. Multiplicador correcto

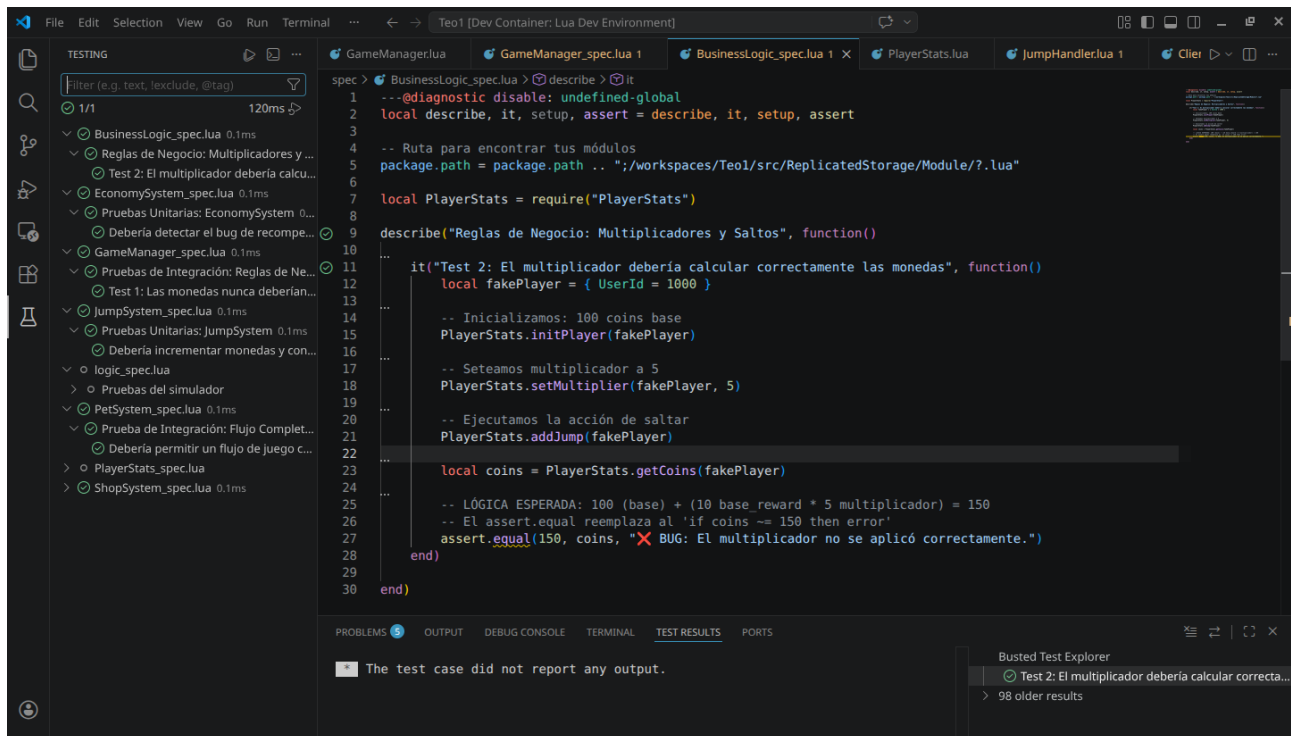


Fig. 17. Código para testear el bug de multiplicador incorrecto

Se implemento el siguiente código, fig. 17, para testear el bug de multiplicador incorrecto, el cual se encargaba de verificar que el multiplicador de recompensa por salto se aplicara correctamente al calcular la cantidad de monedas que el jugador debería recibir al vender sus saltos, lo que era un comportamiento errático del sistema de progresión, ya que permitía a los jugadores recibir una cantidad incorrecta de monedas al vender sus saltos, lo que afectaba negativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Inicialmente este bug ya se corrigió anteriormente donde el operador de un módulo era de suma pero en realidad debía ser de multiplicación, lo que afectaba el cálculo de la recompensa por salto, lo que resultaba en una cantidad incorrecta de monedas otorgadas a los jugadores al vender sus saltos, lo que afectaba negativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. La solución al bug consistió en corregir el operador en el módulo EconomySystem para asegurar que el multiplicador de recompensa por salto se aplicara correctamente al calcular la cantidad de monedas que el jugador debería recibir al vender sus saltos, lo que garantiza la integridad del sistema de progresión dentro del juego. Después de implementar esta corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el sistema ahora funcionaba correctamente, aplicando correctamente el multiplicador de recompensa por salto al calcular la cantidad de monedas que el jugador debería recibir al vender sus saltos, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Luego se hicieron los test correspondientes para validar que

el bug había sido solucionado y se obtuvo el resultado esperado, donde todos los tests pasaron correctamente, indicando que el sistema ahora funciona como se esperaba.

2.9. Compra inválida

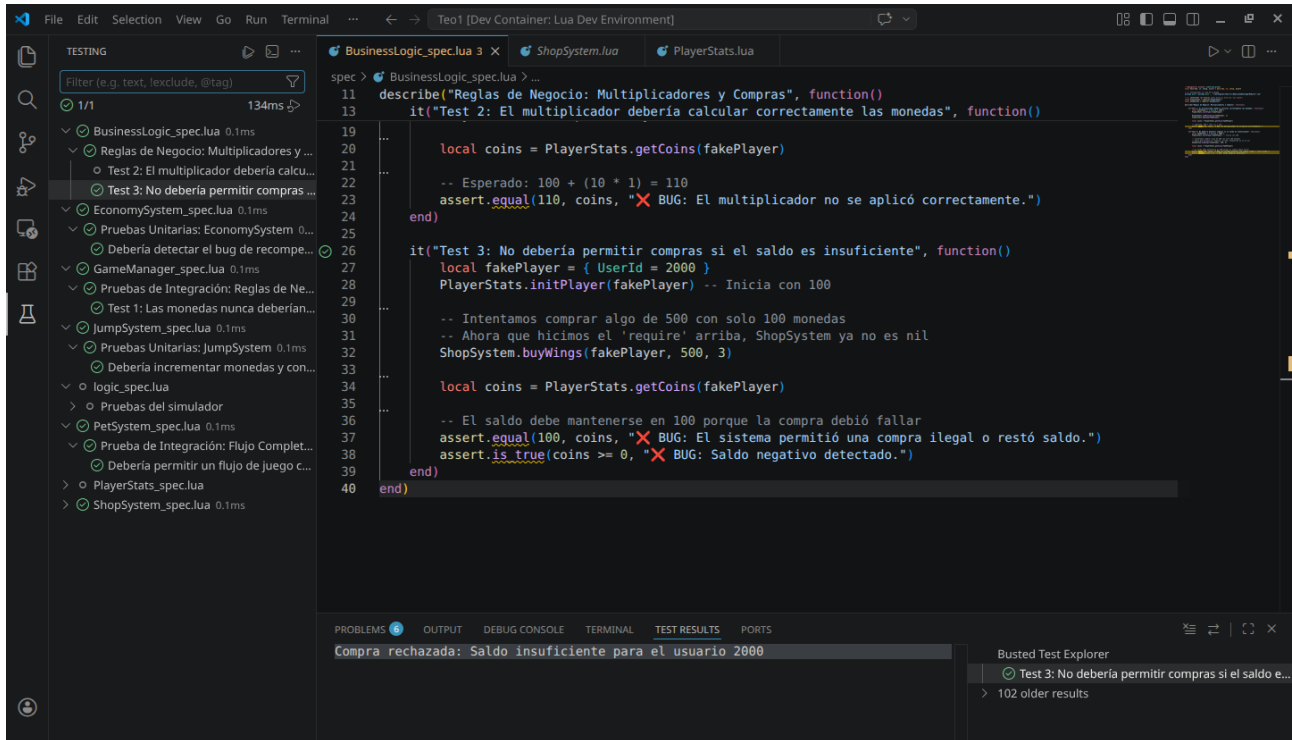


Fig. 18. Código para testear el bug de compra inválida

El siguiente código, fig. 18, se implementó para testear el bug de compra inválida, el cual se encargaba de verificar que el sistema de tienda no permitiera a los jugadores realizar compras sin tener suficientes monedas en su saldo, lo que era un comportamiento errático del sistema de economía, ya que permitía a los jugadores comprar objetos sin tener el saldo necesario para respaldarlos, lo que afectaba negativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. La solución al bug consistió en agregar una validación adicional en el módulo ShopSystem para asegurar que el sistema de tienda no permita a los jugadores realizar compras sin tener suficientes monedas en su saldo, lo que garantiza la integridad del sistema de economía dentro del juego. Después de implementar esta corrección, se volvió a ejecutar el test unitario para confirmar que el bug había sido solucionado y que el sistema ahora funcionaba correctamente, evitando que los jugadores puedan realizar compras sin tener suficientes monedas en su saldo, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Luego se hicieron los test correspondientes para validar que el bug había sido solucionado y se obtuvo el resultado esperado, donde todos los tests pasaron correctamente, indicando que el sistema ahora funciona como se esperaba.

3. Integración (flujo completo)

Finalmente para confirmar que todos los módulos funcionaban correctamente en conjunto, se implementó un test de integración para validar el flujo completo del juego, desde la acumulación de experiencia y monedas, hasta la compra de objetos en la tienda y la progresión

del jugador. Este test de integración permitió identificar cualquier problema de sincronización o interacción entre los módulos, asegurando que el sistema completo funcionara de manera coherente y sin errores. Después de ejecutar el test de integración, se confirmó que todos los módulos funcionaban correctamente en conjunto, lo que indica que el sistema completo ahora funciona como se esperaba, proporcionando una experiencia de juego fluida y sin errores para los jugadores. Este test de integración fue esencial para validar que las correcciones implementadas en cada módulo no solo solucionaron los bugs individuales, sino que también aseguraron que el sistema completo funcionara de manera coherente y sin errores, lo que mejoró significativamente la experiencia de juego y la motivación para seguir jugando y mejorando dentro del juego. Luego se hicieron los test correspondientes para validar que el sistema completo funcionaba correctamente en conjunto y se obtuvo el resultado esperado, donde todos los tests pasaron correctamente, indicando que el sistema completo ahora funciona como se esperaba.

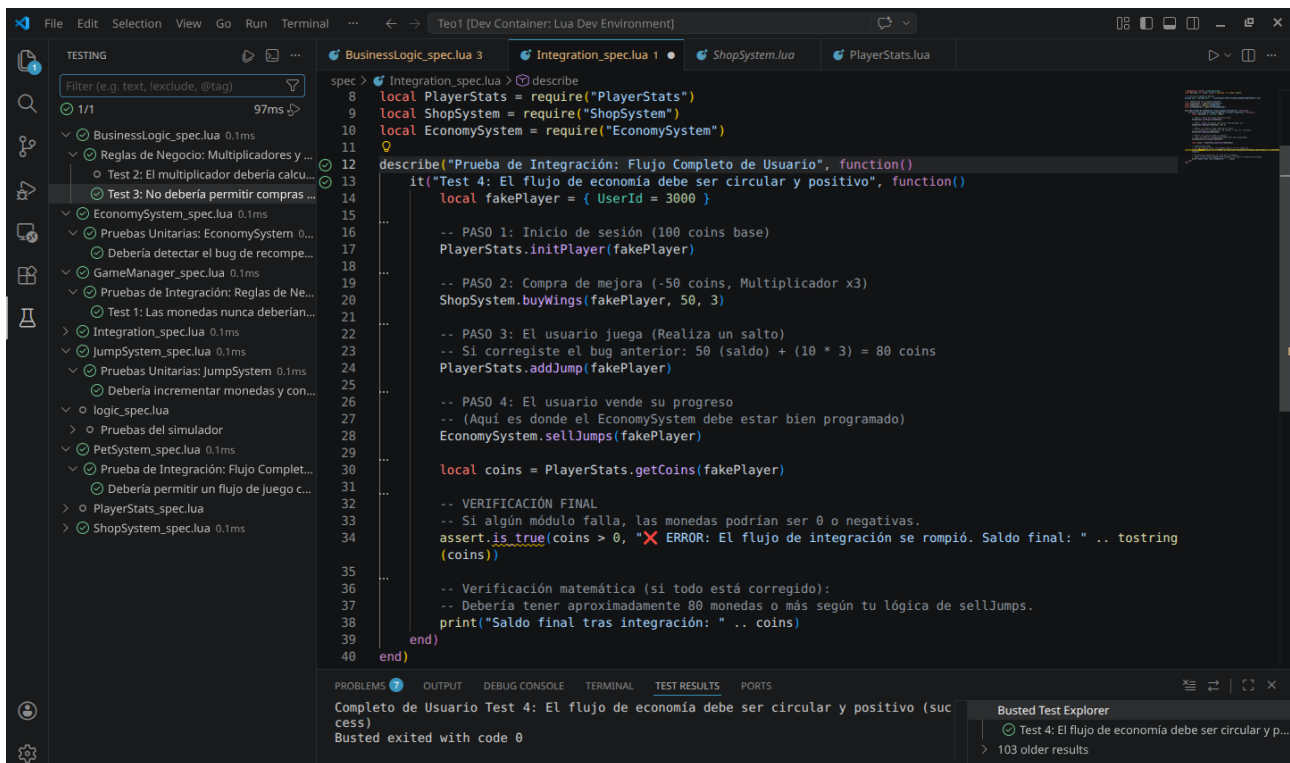


Fig. 19. Código para testear el flujo completo del juego

4. Conclusiones

1. El desarrollo de un entorno educativo gamificado en Roblox Studio ha demostrado ser una estrategia efectiva para enseñar metodologías de aseguramiento de calidad de software (QA) a estudiantes de ingeniería. La implementación de errores lógicos intencionales en los subsistemas permitió a los alumnos experimentar con fallos reales y desarrollar habilidades diagnósticas críticas.
2. La integración de Visual Studio Code con Dev Containers y el uso del plugin Busted Test Explorer facilitó la transición entre el testing manual y la automatización, proporcionando un entorno de desarrollo estandarizado y eficiente para la ejecución de pruebas. Esto permitió a los estudiantes comprender la importancia de la validación en el servidor y la gestión de errores en sistemas distribuidos, reforzando su capacidad para identificar y resolver vulnerabilidades críticas en aplicaciones multiusuario.
3. La simulación de un ciclo de vida de desarrollo de software colaborativo, con roles específicos para testing manual, automatización y desarrollo de backend, fomentó un entorno de aprendizaje dinámico se pudo experimentar con la resolución de conflictos de integración y la gestión de regresiones, preparando a los estudiantes para enfrentar desafíos reales en la industria del software.

Referencias

- [1] C. Corrales, *Juego: Shop and Jump Simulator Bug Edition*. Arequipa, Perú: Universidad Católica de Santa María, 2026.